

Questão 01

```
func max_pooling(pmapas, s)
```

```
    N, M, P = size(pmapas)
```

```
    para k = 0 até P-1
```

```
        para i = 0 até (N//s)-1
```

```
            para j = 0 até (M//s)-1
```

```
                maximo = 0
```

```
                para x = i*s até (i*s)+s-1
```

```
                    para y = j*s até (j*s)+s-1
```

```
                        se pmapas[x,y,k] > maximo
```

```
                            maximo = pmapas[x,y,k]
```

```
                pmapas_res[i,j,k] = maximo
```

```
    returne pmapas_res
```

comentário

// calcula a divisão inteira

Questão 02

```
func extract_bbox(Iorig, Ibin)
```

```
    N, M = size(Ibin)
```

```
    xo = ∞; xf = -∞; yo = ∞; yf = -∞
```

```
    para i = 0 até N-1
```

```
        para j = 0 até M-1
```

```
            se Ibin[i,j] == 1
```

```
                se i < xo
```

```
                    xo = i
```

```
                se j < yo
```

```
                    yo = j
```

```
                se i > xf
```

```
                    xf = i
```

```
                se j > yf
```

```
                    yf = j
```

```
    h = xf - xo
```

```
    w = yf - yo
```

Agora temos xo, yo, h e w, e extraímos o objeto

```
    para x = 0 até h-1
```

```
        para y = 0 até w-1
```

```
            bbox[x,y] = Iorig[xo+x, yo+y]
```

```
    returne bbox
```

Questão 03

```
func gera_masc_bin(masc_cor)
    N, M, _ = size(masc_cor)
    ERVA[1:N, 1:M] = 0
    CULT[1:N, 1:M] = 0
    para i = 0 até N-1
        para j = 0 até M-1
            se masc_cor
                ERVA[i, j] = 1
            se masc_cor[i, j] = [0, 255, 0]
                CULT[i, j] = 1
    retorne ERVA, CULT
```

Questão 04

```
# Entradas:
# matriz: matriz de dimensão  $N \times M \times P$ 
# filtro: matriz de dimensão  $R \times S \times P$ 
#  $N, M, P, R, S$ : inteiros com  $R$  e  $S$  ímpares ( $R \ll N, S \ll M$ )
# Dimensões do resultado da convolução sem padding
h = N - R + 1
w = M - S + 1
# Inicializar matriz resultado com zeros
res[1:h, 1:w] = 0
para x = 0 até h
    para y = 0 até w
        para fx = 0 até R-1
            para fy = 0 até S-1
                para p = 0 até P-1
                    res[x, y] += matriz[x+fx, y+fy, p] * filtro[fx, fy, p]
# res guardará o resultado
```

Questão 05

Entradas:

img: imagem em níveis de cinza

dx: inteiro, representando o deslocamento na linha

dy: " " " " na coluna

$N, M = \text{size}(\text{img})$

$\text{COOC}[1:256, 1:256] = 0$

para $x = 0$ até $N-1$

para $y = 0$ até $M-1$

se $x+dx \geq 0$ AND $x+dx < N$ AND $y+dy \geq 0$ AND $y+dy < M$

$\text{COOC}[\text{img}[x,y], \text{img}[x+dx, y+dy]] += 1$

Questão 06

Entradas:

img_gt: máscara de ground-truth

img_pred: máscara de predição

N, M : dimensões, que são iguais para ambos img_gt e img_pred

$\text{contIntersec} = 0$

$\text{contUnion} = 0$

para $x = 0$ até $N-1$

para $y = 0$ até $M-1$

$\text{contUnion} += \text{img_pred}[x,y] \text{ OR } \text{img_gt}[x,y]$

$\text{contIntersec} += \text{img_pred}[x,y] \text{ AND } \text{img_gt}[x,y]$

$\text{IoU} = \text{contUnion} / \text{contIntersec}$